

Practical 1: Robust Java Calculator

Aim

To implement a robust Java calculator program that captures user input dynamically, processes mathematical expressions using conditional logic, and ensures efficient error handling.

Objectives

- Understand the use of Scanner class for dynamic user input in Java.
- Implement conditional logic using if-else statements for operator selection.
- Handle divide-by-zero and invalid operator errors gracefully.
- Demonstrate modular design by separating calculation logic into a static method.

Code

```
import java.util.Scanner;
public class Calculator {
    static double calculate(double a, double b, char op) {
        if (op == '+') {
            return a + b;
        }
        else if (op == '-') {
            return a - b;
        }
        else if (op == '*') {
            return a * b;
        }
        else if (op == '/') {
            if (b == 0) {
                System.out.println("Error: Cannot divide by zero");
                return 0;
            }
            return a / b;
        }
        else {
            System.out.println("Invalid operator");
            return 0;
        }
    }
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter first number: ");
        double num1 = sc.nextDouble();
        System.out.print("Enter second number: ");
        double num2 = sc.nextDouble();
        System.out.print("Enter operator (+, -, *, /): ");
        char operator = sc.next().charAt(0);
        double result = calculate(num1, num2, operator);
        System.out.println("Result: " + result);
        sc.close();
        System.out.println("Calculator Closed");
    }
}
```

• Output:

```
Enter first number: 88
Enter second number: 45
Enter operator (+, -, *, /): +
Result: 133.0
Calculator Closed
```

Viva Questions & Answers

Q1. What is the role of the Scanner class in this program?

Ans: Scanner is used to read user input from the standard input stream (System.in). It provides methods like nextDouble() and next() to read numeric and character input respectively at runtime.

Q2. Why is the calculate() method declared as static?

Ans: The calculate() method is static so it can be called directly from main() without creating an instance of the Calculator class. Static methods belong to the class itself, not to any particular object.

Q3. What happens when the user enters 0 as the divisor?

Ans: The program checks if `b == 0` before performing division. If true, it prints 'Error: Cannot divide by zero' and returns 0, preventing a runtime ArithmeticException from being thrown.

Q4. How does `sc.next().charAt(0)` work for reading the operator?

Ans: `sc.next()` reads the next token as a String. `charAt(0)` extracts the first character of that string, giving the operator as a char. Java's Scanner has no `nextChar()` method, so this is the standard approach.

Q5. How would you extend this program to support continuous calculations using a loop?

Ans: Wrap the input and calculation logic inside a do-while loop and prompt the user to continue (e.g., enter 'y/n'). This allows multiple calculations in a single run without restarting the program.

Practical 2: E-Commerce Order Processing

Aim

To develop an E-Commerce order processing system using multiple constructors, dynamic cost computation, and discount policies.

Objectives

- Understand constructor overloading in Java.
- Accept user input dynamically and compute total order cost.
- Apply discount policies based on total order value conditions.
- Generate a formatted invoice summarizing the entire purchase.

Code

```
import java.util.Scanner;
class Product {
    int productId; String productName; double price; int quantity;
    Product() { productId=0; productName="NA"; price=0.0; quantity=0; }
    Product(int id,String name) { productId=id; productName=name; price=0; quantity=0; }
    Product(int id,String name,double p,int q) {
        productId=id; productName=name; price=p; quantity=q;
    }
    double calculateTotal() { return price*quantity; }
}
public class Main {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        Product p1=new Product(101,"Laptop",50000.0,1);
        System.out.println("Enter Product ID:"); int id=sc.nextInt(); sc.nextLine();
        System.out.println("Enter Product Name:"); String name=sc.nextLine();
        System.out.println("Enter Product Price:"); double price=sc.nextDouble();
        System.out.println("Enter Quantity:"); int quantity=sc.nextInt();
        Product p2=new Product(id,name,price,quantity);
        double totalAmount=p1.calculateTotal()+p2.calculateTotal();
        double discount=0.0;
        if(totalAmount>=60000) discount=totalAmount*0.10;
        else if(totalAmount>=30000) discount=totalAmount*0.05;
        double finalAmount=totalAmount-discount;
        System.out.println("\n----- INVOICE -----");
        System.out.println("Product ID: "+p1.productId);
        System.out.println("Product Name: "+p1.productName);
        System.out.println("Price: "+p1.price+" Quantity: "+p1.quantity);
        System.out.println("Total: "+p1.calculateTotal());
        System.out.println("-----");
        System.out.println("Product ID: "+p2.productId);
        System.out.println("Product Name: "+p2.productName);
        System.out.println("Price: "+p2.price+" Quantity: "+p2.quantity);
        System.out.println("Total: "+p2.calculateTotal());
        System.out.println("-----");
        System.out.println("Total Amount: "+totalAmount);
        System.out.println("Discount: "+discount);
        System.out.println("Final Payable Amount: "+finalAmount);
        sc.close();
    }
}
```

• **Output:**

```
Enter Product ID: 101
Enter Product Name: laptop
Enter Product Price: 78000
Enter Quantity: 2

----- INVOICE -----
Product ID: 101  Product Name: Laptop  Price: 50000.0  Quantity: 1  Total: 50000.0
-----
Product ID: 101  Product Name: laptop  Price: 78000.0  Quantity: 2  Total: 156000.0
-----
Total Amount: 206000.0
Discount: 20600.0
Final Payable Amount: 185400.0
```

Viva Questions & Answers

Q1. What is constructor overloading and why is it useful?

Ans: Constructor overloading means having multiple constructors with different parameter lists in the same class. It allows objects to be initialized in different ways depending on the data available, providing flexibility.

Q2. What is the default constructor and when is it invoked?

Ans: A default constructor takes no parameters. It is invoked when an object is created without passing arguments, e.g., `new Product()`. If no constructor is defined, Java provides one automatically.

Q3. What is the purpose of `sc.nextLine()` after `sc.nextInt()`?

Ans: `sc.nextInt()` reads the integer but leaves the newline character in the buffer. Calling `sc.nextLine()` consumes that leftover newline so the subsequent `sc.nextLine()` for string input works correctly.

Q4. How would you add a 15% discount tier above Rs. 1,00,000?

Ans: Add a new condition at the top: `if(totalAmount >= 100000) discount = totalAmount * 0.15;` before the existing 60000 check, since if-else checks conditions in order.

Q5. What is the difference between a constructor and a method?

Ans: A constructor has the same name as the class, no return type, and is called automatically when an object is created. A method has a return type, can have any name, and must be called explicitly.

Practical 3: Method Overloading

Aim

To demonstrate method overloading in Java for computing power and absolute values for different data types.

Objectives

- Understand the concept of method overloading in Java.
- Implement overloaded methods for int and double data types.
- Compare user-defined overloaded methods with built-in Math class methods.
- Demonstrate compile-time polymorphism through method overloading.

Code

```
class OverloadDemo {
    static int power(int a, int b) {
        int result=1;
        for(int i=0;i<b;i++) result*=a;
        return result;
    }
    static double power(double a, double b) { return Math.pow(a,b); }
    static int absolute(int a) { return (a<0)?-a:a; }
    static double absolute(double a) { return (a<0)?-a:a; }
    public static void main(String[] args) {
        int x=-5, y=3; double p=-4.5, q=2.0;
        System.out.println("Power (int): " + power(x,y));
        System.out.println("Power (double): " + power(p,q));
        System.out.println("Absolute (int): " + absolute(x));
        System.out.println("Absolute (double): " + absolute(p));
        System.out.println("Math.pow(): " + Math.pow(x,y));
        System.out.println("Math.abs(int): " + Math.abs(x));
        System.out.println("Math.abs(double): " + Math.abs(p));
    }
}
```

• Output:

```
Power (int): -125
Power (double): 20.25
Absolute (int): 5
Absolute (double): 4.5
Math.pow(): -125.0
Math.abs(int): 5
Math.abs(double): 4.5
```

Viva Questions & Answers

Q1. What is method overloading and how does Java differentiate between overloaded methods?

Ans: Method overloading allows multiple methods with the same name but different parameter types or counts. Java differentiates them at compile time based on the method signature (parameter list).

Q2. Can two methods have the same name and same parameters but different return types?

Ans: No. Return type alone is not enough to differentiate overloaded methods. Java would throw a compile-time error since it cannot determine which method to call based on return type.

Q3. What is the difference between method overloading and method overriding?

Ans: Overloading occurs in the same class with same method name but different parameters (compile-time polymorphism). Overriding occurs in subclass with same method name and parameters as superclass (runtime polymorphism).

Q4. What does Math.pow() return and what are its parameter types?

Ans: Math.pow(double a, double b) returns a double value representing a raised to the power b. Both parameters are of type double.

Q5. Is method overloading compile-time or runtime polymorphism?

Ans: Method overloading is compile-time (static) polymorphism. The correct method to call is resolved at compile time based on the argument types passed.

Practical 4: Library Management System

Aim

To implement a Library Management System in Java using static fields and methods.

Objectives

- Understand the use of static variables and methods in Java.
- Implement OOP concepts such as classes, methods, and objects.
- Manage book records including issuing and returning operations.
- Track total available books using a static counter field.

Code

```
import java.util.Scanner;
class Book {
    int bookId; String title; String author; boolean isIssued;
    static int totalBooks = 0;
    Book(int bookId, String title, String author) {
        this.bookId=bookId; this.title=title; this.author=author;
        this.isIssued=false; totalBooks++;
    }
    void issueBook() {
        if (!isIssued) { isIssued=true; totalBooks--;
            System.out.println("Book issued successfully.");
        } else System.out.println("Book already issued.");
    }
    void returnBook() {
        if (isIssued) { isIssued=false; totalBooks++;
            System.out.println("Book returned successfully.");
        } else System.out.println("Book was not issued.");
    }
    void displayBookDetails() {
        System.out.println("\nBook ID: "+bookId);
        System.out.println("Title: "+title);
        System.out.println("Author: "+author);
        System.out.println("Status: "+(isIssued?"Issued":"Available"));
    }
    static void displayTotalBooks() {
        System.out.println("\nTotal Available Books in Library: "+totalBooks);
    }
}
public class LibraryManagementSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Book b1 = new Book(101,"Java Programming","James Gosling");
        Book b2 = new Book(102,"Data Structures","Mark Weiss");
        int choice;
        do {
            System.out.println("\n--- Library Management System ---");
            System.out.println("1.View Details 2.Issue 3.Return 4.Total 5.Exit");
            System.out.print("Enter choice: "); choice=sc.nextInt();
            switch(choice) {
                case 1: b1.displayBookDetails(); b2.displayBookDetails(); break;
                case 2:
                    System.out.print("Enter Book ID to issue: "); int iid=sc.nextInt();
                    if(iid==101) b1.issueBook(); else if(iid==102) b2.issueBook();
                    else System.out.println("Invalid Book ID"); break;
                case 3:
                    System.out.print("Enter Book ID to return: "); int rid=sc.nextInt();
                    if(rid==101) b1.returnBook(); else if(rid==102) b2.returnBook();
                    else System.out.println("Invalid Book ID"); break;
                case 4: Book.displayTotalBooks(); break;
                case 5: System.out.println("Thank you!"); break;
                default: System.out.println("Invalid choice");
            }
        } while(choice!=5);
    }
}
```

```

        sc.close();
    }
}

```

• Output:

```

--- Library Management System ---
1.View Details 2.Issue 3.Return 4.Total 5.Exit
Enter choice: 1
Book ID: 101  Title: Java Programming  Author: James Gosling  Status: Available
Book ID: 102  Title: Data Structures  Author: Mark Weiss  Status: Available
Enter choice: 2  Enter Book ID to issue: 102
Book issued successfully.
Enter choice: 3  Enter Book ID to return: 102
Book returned successfully.
Enter choice: 4
Total Available Books in Library: 2
Enter choice: 5  Thank you!

```

Viva Questions & Answers

Q1. What is a static variable and how does it differ from an instance variable?

Ans: A static variable belongs to the class itself and is shared by all objects, while an instance variable has a separate copy for each object. Here, totalBooks is static so all Book objects share the same counter.

Q2. Why is totalBooks declared as static in the Book class?

Ans: Because totalBooks must reflect the count across all book objects, not just one. A static variable is shared across all instances, making it ideal for tracking a global count.

Q3. What is the difference between static and non-static methods?

Ans: Static methods belong to the class and can be called without creating an object (e.g., Book.displayTotalBooks()). Non-static methods require an object instance to be called.

Q4. What happens if totalBooks is not decremented when a book is issued?

Ans: The count would remain incorrect, showing more books as available than actually are. The counter would not reflect the true number of books available in the library.

Q5. How does the do-while loop help in keeping the menu active?

Ans: The do-while loop executes the menu at least once and continues until the user enters 5 (Exit). This ensures the menu is always shown before checking the exit condition.

Practical 5: Array Operations

Aim

To perform various operations on arrays in Java including display, max/min, sum, average, and linear search.

Objectives

- Understand array declaration and initialization in Java.
- Implement traversal and searching algorithms on arrays.
- Calculate statistical values such as sum, average, max, and min.
- Use Scanner class for dynamic user input at runtime.

Code

```
import java.util.Scanner;
public class ArrayOperations {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter the number of elements: ");
        int n = sc.nextInt();
        int[] arr = new int[n];
        System.out.println("Enter the elements:");
        for(int i=0;i<n;i++) arr[i]=sc.nextInt();
        System.out.println("\nArray elements are:");
        for(int i=0;i<n;i++) System.out.print(arr[i]+" ");
        int max=arr[0], min=arr[0], sum=0;
        for(int i=0;i<n;i++) {
            if(arr[i]>max) max=arr[i];
            if(arr[i]<min) min=arr[i];
            sum+=arr[i];
        }
        double average=(double)sum/n;
        System.out.println("\n\nMaximum element: "+max);
        System.out.println("Minimum element: "+min);
        System.out.println("Sum of elements: "+sum);
        System.out.println("Average of elements: "+average);
        System.out.print("\nEnter element to search: ");
        int search=sc.nextInt(); boolean found=false;
        for(int i=0;i<n;i++) {
            if(arr[i]==search) {
                System.out.println("Element found at index: "+i);
                found=true; break;
            }
        }
        if(!found) System.out.println("Element not found in the array.");
        sc.close();
    }
}
```

• Output:

```
Enter the number of elements: 5
Enter the elements: 10 20 30 40 50
Array elements are: 10 20 30 40 50
Maximum element: 50
Minimum element: 10
Sum of elements: 150
Average of elements: 30.0
Enter element to search: 60
Element not found in the array.
```

Viva Questions & Answers**Q1. What is the default value of an int array element in Java?**

Ans: The default value of an int array element in Java is 0. Java initializes all array elements to default values when the array is created.

Q2. What is the time complexity of linear search?

Ans: The time complexity of linear search is $O(n)$, where n is the number of elements. In the worst case, the element is at the last position or not present at all.

Q3. How is an array different from an ArrayList in Java?

Ans: An array has a fixed size set at creation time, while ArrayList is dynamic and can grow or shrink. Arrays can store primitives; ArrayList stores only objects. ArrayList is part of the Collections framework.

Q4. What exception is thrown if you access an invalid array index?

Ans: ArrayIndexOutOfBoundsException is thrown when you try to access an index that is negative or greater than or equal to the array length.

Q5. Why is (double) cast used when calculating average?

Ans: Because sum and n are both integers; integer division would truncate the decimal part. Casting sum to double ensures floating-point division, giving an accurate average.

Practical 6: Single Inheritance

Aim

To demonstrate single inheritance in Java where a subclass extends a superclass and calls its methods.

Objectives

- Understand the IS-A relationship and single inheritance in Java.
- Use the super keyword to invoke superclass constructors and methods.
- Override superclass methods in the subclass for specific behavior.
- Demonstrate code reuse and extensibility through inheritance.

Code

```
class Animal {
    String name; String sound;
    Animal(String name,String sound){ this.name=name; this.sound=sound; }
    void eat()          { System.out.println(name+" is eating."); }
    void sleep()        { System.out.println(name+" is sleeping."); }
    void makeSound() { System.out.println(name+" says: "+sound); }
    void displayInfo() {
        System.out.println("Animal Name : "+name);
        System.out.println("Sound      : "+sound);
    }
}

class Dog extends Animal {
    String breed;
    Dog(String name,String breed){ super(name,"Woof"); this.breed=breed; }
    void fetch(){ System.out.println(name+" is fetching the ball!"); }
    void displayInfo(){ super.displayInfo(); System.out.println("Breed      : "+breed); }
}

public class InheritanceDemo {
    public static void main(String[] args) {
        System.out.println("--- Superclass Object ---");
        Animal a=new Animal("Cat","Meow");
        a.displayInfo(); a.eat(); a.sleep(); a.makeSound();
        System.out.println("\n--- Subclass Object ---");
        Dog d=new Dog("Bruno","Labrador");
        d.displayInfo(); d.eat(); d.sleep(); d.makeSound(); d.fetch();
    }
}
```

• Output:

```
--- Superclass Object ---
Animal Name : Cat  Sound : Meow
Cat is eating.  Cat is sleeping.  Cat says: Meow
--- Subclass Object ---
Animal Name : Bruno  Sound : Woof  Breed : Labrador
Bruno is eating.  Bruno is sleeping.  Bruno says: Woof
Bruno is fetching the ball!
```

Viva Questions & Answers

Q1. What is inheritance and what are its advantages in Java?

Ans: Inheritance allows a subclass to acquire properties and methods of a superclass. Advantages include code reuse, method overriding, and establishing an IS-A relationship between classes.

Q2. What does the super keyword do in a subclass constructor?

Ans: super() calls the constructor of the immediate superclass. It must be the first statement in the subclass constructor. Here, Dog calls super(name, "Woof") to initialize the Animal part.

Q3. Can a subclass access private members of the superclass?

Ans: No. Private members are not accessible outside their class, even by subclasses. They can only be accessed via public or protected getter/setter methods defined in the superclass.

Q4. What is method overriding and how does it differ from overloading?

Ans: Overriding redefines a superclass method in the subclass with the same signature — it is runtime polymorphism. Overloading defines multiple methods with the same name but different parameters in the same class — it is compile-time polymorphism.

Q5. What is the role of the Object class in Java?

Ans: Object is the root class of all Java classes. Every class implicitly extends Object. It provides common methods like toString(), equals(), hashCode(), and getClass() to all Java objects.

Practical 7: Interface and Polymorphism

Aim

To implement an interface in Java and create multiple classes implementing it, demonstrating runtime polymorphism.

Objectives

- Understand interfaces and how they enforce a contract on classes.
- Implement the Shape interface in Circle, Rectangle, and Triangle classes.
- Demonstrate runtime polymorphism using interface-type array references.
- Apply Heron's formula for computing triangle area.

Code

```
interface Shape { double area(); double perimeter(); void display(); }
class Circle implements Shape {
    double radius;
    Circle(double r){ this.radius=r; }
    public double area() { return Math.PI*radius*radius; }
    public double perimeter() { return 2*Math.PI*radius; }
    public void display() {
        System.out.println("Shape: Circle");
        System.out.printf(" Radius      : %.2f\n",radius);
        System.out.printf(" Area        : %.2f\n",area());
        System.out.printf(" Perimeter  : %.2f\n",perimeter());
    }
}
class Rectangle implements Shape {
    double length,width;
    Rectangle(double l,double w){ length=l; width=w; }
    public double area() { return length*width; }
    public double perimeter() { return 2*(length+width); }
    public void display() {
        System.out.println("Shape: Rectangle");
        System.out.printf(" Length    : %.2f Width : %.2f\n",length,width);
        System.out.printf(" Area      : %.2f\n",area());
        System.out.printf(" Perimeter : %.2f\n",perimeter());
    }
}
class Triangle implements Shape {
    double a,b,c;
    Triangle(double a,double b,double c){ this.a=a; this.b=b; this.c=c; }
    public double area() { double s=(a+b+c)/2; return Math.sqrt(s*(s-a)*(s-b)*(s-c)); }
    public double perimeter() { return a+b+c; }
    public void display() {
        System.out.println("Shape: Triangle");
        System.out.printf(" Sides     : %.2f,%.2f,%.2f\n",a,b,c);
        System.out.printf(" Area      : %.2f\n",area());
        System.out.printf(" Perimeter : %.2f\n",perimeter());
    }
}
public class ShapeDemo {
    public static void main(String[] args) {
        Shape[] shapes={ new Circle(7), new Rectangle(5,8), new Triangle(3,4,5) };
        System.out.println("--- Shape Details ---");
        for(Shape s:shapes){ s.display(); System.out.println(); }
    }
}
```

• Output:

```
--- Shape Details ---  
Shape: Circle    Radius: 7.00  Area: 153.94  Perimeter: 43.98  
Shape: Rectangle Length: 5.00  Width: 8.00  Area: 40.00  Perimeter: 26.00  
Shape: Triangle  Sides: 3.00,4.00,5.00  Area: 6.00  Perimeter: 12.00
```

Viva Questions & Answers**Q1. What is an interface in Java and how does it differ from an abstract class?**

Ans: An interface is a contract with abstract method declarations only (pre-Java 8). All methods are implicitly public and abstract. An abstract class can have both abstract and concrete methods, constructors, and instance variables. A class can implement multiple interfaces but extend only one class.

Q2. Can an interface have instance variables?

Ans: No. Variables declared inside an interface are implicitly public, static, and final (constants). They cannot be instance variables since interfaces cannot be instantiated.

Q3. What is runtime polymorphism and how is it achieved?

Ans: Runtime polymorphism means the method to be called is determined at runtime based on the actual object type, not the reference type. It is achieved through method overriding and interface/superclass references pointing to subclass objects.

Q4. What happens if a class implements an interface but doesn't define all its methods?

Ans: The class must be declared abstract. If it is not abstract, the compiler throws an error saying the class must implement all abstract methods of the interface.

Q5. Can a class implement multiple interfaces? Can it extend multiple classes?

Ans: Yes, a class can implement multiple interfaces (e.g., class A implements B, C). However, Java does not support multiple class inheritance — a class can extend only one superclass.

Practical 8: ATM Machine Simulation

Aim

To simulate an ATM machine in Java using exception handling with try, catch, and finally blocks.

Objectives

- Understand Java exception handling using try-catch-finally.
- Throw and handle ArithmeticException and IllegalArgumentException.
- Ensure the application continues running smoothly after handling exceptions.
- Implement deposit, withdraw, and balance-check functionalities.

Code

```
import java.util.Scanner;
class ATM {
    private double balance; private String accountHolder;
    ATM(String name,double bal){ accountHolder=name; balance=bal; }
    void checkBalance() {
        System.out.printf("Account Holder : %s\n",accountHolder);
        System.out.printf("Balance          : Rs. %.2f\n",balance);
    }
    void deposit(double amount) {
        try {
            if(amount<=0) throw new IllegalArgumentException("Deposit must be positive.");
            balance+=amount;
            System.out.printf("Rs. %.2f deposited successfully.\n",amount);
        } catch(IllegalArgumentException e) {
            System.out.println("Deposit Error: "+e.getMessage());
        } finally { System.out.println("Deposit operation completed."); }
    }
    void withdraw(double amount) {
        try {
            if(amount<=0) throw new IllegalArgumentException("Withdrawal must be positive.");
            if(amount>balance)
                throw new ArithmeticException("Insufficient funds. Available: Rs."+balance);
            balance-=amount;
            System.out.printf("Rs. %.2f withdrawn successfully.\n",amount);
        } catch(IllegalArgumentException e) {
            System.out.println("Input Error: "+e.getMessage());
        } catch(ArithmeticException e) {
            System.out.println("Transaction Error: "+e.getMessage());
        } finally { System.out.println("Withdrawal operation completed."); }
    }
}
public class ATMSimulation {
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in);
        ATM atm=new ATM("Rahul Sharma",10000.00); int choice;
        do {
            System.out.println("\n--- ATM Menu ---");
            System.out.println("1.Check Balance 2.Deposit 3.Withdraw 4.Exit");
            System.out.print("Enter choice: "); choice=sc.nextInt();
            switch(choice) {
                case 1: atm.checkBalance(); break;
                case 2: System.out.print("Enter deposit amount: ");
                        atm.deposit(sc.nextDouble()); break;
                case 3: System.out.print("Enter withdrawal amount: ");
                        atm.withdraw(sc.nextDouble()); break;
                case 4: System.out.println("Thank you. Goodbye!"); break;
                default: System.out.println("Invalid choice.");
            }
        } while(choice!=4);
        sc.close();
    }
}
```

• Output:

```

--- ATM Menu --- Enter choice: 1
Account Holder : Rahul Sharma Balance : Rs. 10000.00
--- ATM Menu --- Enter choice: 2 Enter deposit amount: 5000
Rs. 5000.00 deposited successfully. Deposit operation completed.
--- ATM Menu --- Enter choice: 3 Enter withdrawal amount: 20000
Transaction Error: Insufficient funds. Available: Rs.15000.0
Withdrawal operation completed.
--- ATM Menu --- Enter choice: 3 Enter withdrawal amount: -500
Input Error: Withdrawal must be positive. Withdrawal operation completed.
--- ATM Menu --- Enter choice: 4 Thank you. Goodbye!

```

Viva Questions & Answers

Q1. What is the purpose of the finally block in Java?

Ans: The finally block always executes regardless of whether an exception occurred or was caught. It is used for cleanup tasks like closing resources, printing completion messages, or releasing locks.

Q2. What is the difference between checked and unchecked exceptions?

Ans: Checked exceptions (e.g., IOException) must be declared or handled at compile time. Unchecked exceptions (e.g., ArithmeticException, IllegalArgumentException) are subclasses of RuntimeException and don't require mandatory handling.

Q3. Why is ArithmeticException used for insufficient funds?

Ans: ArithmeticException is an unchecked runtime exception suitable for arithmetic/logical violations like an invalid financial operation. A custom exception would be more professional but ArithmeticException conveys the same meaning here.

Q4. What happens if an exception is thrown inside the finally block?

Ans: If an exception is thrown inside finally, it overrides any exception thrown in the try or catch block. The original exception is lost and the new exception from finally propagates up the call stack.

Q5. Can we have multiple catch blocks for a single try? In what order?

Ans: Yes. Multiple catch blocks are allowed. They must be ordered from most specific (child) to most general (parent). If a parent exception class is caught first, child-specific catch blocks become unreachable and cause a compile error.

Practical 9: Multithreading Chat Simulation

Aim

To simulate real-time user interaction using multithreading with synchronization and the Producer-Consumer pattern.

Objectives

- Understand thread creation by extending the Thread class.
- Use synchronized methods for thread-safe shared resource access.
- Apply wait() and notifyAll() for inter-thread communication.
- Implement the Producer-Consumer design pattern in a chat room.

Code

```
import java.util.LinkedList; import java.util.Queue;
class ChatRoom {
    private final Queue<String> messageQueue=new LinkedList<>();
    private final int MAX_MESSAGES=5; private boolean active=true;
    synchronized void sendMessage(String sender,String msg) throws InterruptedException {
        while(messageQueue.size()>=MAX_MESSAGES) {
            System.out.println "["+sender+"] Waiting - chat room full..."); wait();
        }
        String formatted="["+sender+"]: "+msg;
        messageQueue.add(formatted);
        System.out.println("SENT    >> "+formatted); notifyAll();
    }
    synchronized String readMessage(String reader) throws InterruptedException {
        while(messageQueue.isEmpty()&&active) {
            System.out.println "["+reader+"] Waiting for messages..."); wait(1000);
        }
        if(!messageQueue.isEmpty()) {
            String msg=messageQueue.poll();
            System.out.println("READ    << {"+reader+"} "+msg); notifyAll(); return msg;
        }
        return null;
    }
    synchronized void shutdown(){ active=false; notifyAll(); }
    synchronized boolean isActive(){ return active||!messageQueue.isEmpty(); }
}
class UserSender extends Thread {
    private final ChatRoom room; private final String username; private final String[]
messages;
    UserSender(ChatRoom r,String u,String[] m){ room=r; username=u; messages=m; }
    public void run() {
        try {
            for(String msg:messages){ room.sendMessage(username,msg); Thread.sleep(500); }
        } catch(InterruptedException e){ System.out.println(username+" interrupted."); }
    }
}
class Moderator extends Thread {
    private final ChatRoom room; private final String name; private final int total;
    Moderator(ChatRoom r,String n,int t){ room=r; name=n; total=t; }
    public void run() {
        try {
            int count=0;
            while(count<total){ String m=room.readMessage(name); if(m!=null)count++;
Thread.sleep(300); }
        } catch(InterruptedException e){ System.out.println(name+" interrupted."); }
    }
}
public class ChatSimulation {
    public static void main(String[] args) throws InterruptedException {
        ChatRoom room=new ChatRoom();
        String[] m1={"Hey everyone!","Working on a Java project.","Threads are powerful!"};
        String[] m2={"Hello Prathamesh!","Which project?","Multithreading is tricky."};
        UserSender u1=new UserSender(room,"Prathamesh Yewale",m1);
```

```

        UserSender u2=new UserSender(room,"Rahul Sharma",m2);
        Moderator mod=new Moderator(room,"Moderator",6);
        System.out.println("=== Real-Time Chat Simulation Started ===");
        System.out.println("-".repeat(50));
        mod.start(); u1.start(); u2.start();
        u1.join(); u2.join(); mod.join();
        room.shutdown();
        System.out.println("-".repeat(50));
        System.out.println("=== Chat Session Ended ===");
    }
}

```

• Output:

```

=== Real-Time Chat Simulation Started ===
-----
{Moderator} Waiting for messages...
SENT  >> [Prathamesh Yewale]: Hey everyone!
SENT  >> [Rahul Sharma]: Hello Prathamesh!
READ  << {Moderator} [Prathamesh Yewale]: Hey everyone!
READ  << {Moderator} [Rahul Sharma]: Hello Prathamesh!
SENT  >> [Prathamesh Yewale]: Working on a Java project.
SENT  >> [Rahul Sharma]: Which project?
READ  << {Moderator} [Prathamesh Yewale]: Working on a Java project.
READ  << {Moderator} [Rahul Sharma]: Which project?
SENT  >> [Prathamesh Yewale]: Threads are powerful!
SENT  >> [Rahul Sharma]: Multithreading is tricky.
READ  << {Moderator} [Prathamesh Yewale]: Threads are powerful!
READ  << {Moderator} [Rahul Sharma]: Multithreading is tricky.
-----
=== Chat Session Ended ===

```

Viva Questions & Answers

Q1. What is the difference between a process and a thread in Java?

Ans: A process is an independent program in execution with its own memory space. A thread is a lightweight unit of execution within a process, sharing the process's memory. Multiple threads run concurrently within one process.

Q2. Why is the synchronized keyword used in sendMessage and readMessage?

Ans: synchronized ensures that only one thread can execute the method at a time on the same object. This prevents race conditions where multiple threads could corrupt the shared messageQueue simultaneously.

Q3. What is the Producer-Consumer problem and how is it solved here?

Ans: The Producer-Consumer problem involves producers adding items to a shared buffer and consumers removing them, without overflow or underflow. Here, UserSender threads produce messages, Moderator consumes them; wait() pauses when buffer is full/empty and notifyAll() wakes waiting threads.

Q4. What is the difference between wait() and sleep()?

Ans: wait() releases the object's lock and pauses the thread until notified — it must be called from a synchronized context. sleep() pauses the thread for a specified time without releasing any locks.

Q5. What does join() do and why is it used?

Ans: join() makes the calling thread wait until the thread it is called on finishes execution. Here, the main thread calls u1.join(), u2.join(), mod.join() to ensure all threads complete before shutdown() is called.

Practical 10: JDBC Employee Management System

Aim

To create a Java application connecting to a MySQL database using JDBC to perform CRUD operations on employee records.

Objectives

- Understand JDBC architecture and database connectivity in Java.
- Use PreparedStatement with parameterized queries to prevent SQL injection.
- Implement add, view, update, and delete operations on employee records.
- Handle SQL exceptions gracefully using try-catch and try-with-resources.

Code

```
import java.sql.*; import java.util.Scanner;
public class EmployeeManagement {
    static final String URL ="jdbc:mysql://localhost:3306/company_db";
    static final String USER="root", PASS="password";
    static Connection getConnection() throws SQLException {
        return DriverManager.getConnection(URL,USER,PASS);
    }
    static void addEmployee(int id,String name,String dept,double salary) {
        String sql="INSERT INTO employees(emp_id,name,department,salary) VALUES(?,?,?,?)";
        try(Connection con=getConnection(); PreparedStatement ps=con.prepareStatement(sql)) {
            ps.setInt(1,id); ps.setString(2,name); ps.setString(3,dept);
            ps.setDouble(4,salary);
            System.out.println(ps.executeUpdate()>0?"Employee added.":"Failed.");
        } catch(SQLIntegrityConstraintViolationException e) {
            System.out.println("Error: Employee ID "+id+" already exists.");
        } catch(SQLException e){ System.out.println("DB Error: "+e.getMessage()); }
    }
    static void viewEmployees() {
        String sql="SELECT * FROM employees ORDER BY emp_id";
        try(Connection con=getConnection(); PreparedStatement ps=con.prepareStatement(sql);
            ResultSet rs=ps.executeQuery()) {
            System.out.println("-".repeat(55));
            System.out.printf("%-8s %-18s %-14s %s\n","EMP_ID","NAME","DEPARTMENT","SALARY");
            System.out.println("-".repeat(55));
            boolean found=false;
            while(rs.next()) { found=true;
                System.out.printf("%-8d %-18s %-14s %.2f\n",
                    rs.getInt("emp_id"),rs.getString("name"),
                    rs.getString("department"),rs.getDouble("salary"));
            }
            if(!found) System.out.println("No records found.");
            System.out.println("-".repeat(55));
        } catch(SQLException e){ System.out.println("DB Error: "+e.getMessage()); }
    }
    static void updateSalary(int id,double sal) {
        String sql="UPDATE employees SET salary=? WHERE emp_id=?";
        try(Connection con=getConnection(); PreparedStatement ps=con.prepareStatement(sql)) {
            ps.setDouble(1,sal); ps.setInt(2,id);
            System.out.println(ps.executeUpdate()>0?"Salary updated.":"ID "+id+" not
found.");
        } catch(SQLException e){ System.out.println("DB Error: "+e.getMessage()); }
    }
    static void deleteEmployee(int id) {
        String sql="DELETE FROM employees WHERE emp_id=?";
        try(Connection con=getConnection(); PreparedStatement ps=con.prepareStatement(sql)) {
            ps.setInt(1,id);
            System.out.println(ps.executeUpdate()>0?"Employee deleted.":"ID "+id+" not
found.");
        } catch(SQLException e){ System.out.println("DB Error: "+e.getMessage()); }
    }
    public static void main(String[] args) {
        Scanner sc=new Scanner(System.in); int choice;
```

```

do {
    System.out.println("\n=== Employee Management System ===");
    System.out.println("1.Add 2.View 3.Update 4.Delete 5.Exit");
    System.out.print("Enter choice: "); choice=sc.nextInt();
    switch(choice) {
        case 1: System.out.print("ID: "); int id=sc.nextInt(); sc.nextLine();
                System.out.print("Name: "); String nm=sc.nextLine();
                System.out.print("Dept: "); String dp=sc.nextLine();
                System.out.print("Salary: "); addEmployee(id,nm,dp,sc.nextDouble());
        break;

        case 2: viewEmployees(); break;
        case 3: System.out.print("ID: "); int uid=sc.nextInt();
                System.out.print("New Salary: "); updateSalary(uid,sc.nextDouble());
        break;

        case 4: System.out.print("ID: "); deleteEmployee(sc.nextInt()); break;
        case 5: System.out.println("Exiting. Goodbye!"); break;
        default: System.out.println("Invalid choice.");
    }
} while(choice!=5); sc.close();
}
}

```

• Output:

```

=== Employee Management System ===
1.Add 2.View 3.Update 4.Delete 5.Exit   Enter choice: 1
ID: 101  Name: Prathamesh Yewale  Dept: Engineering  Salary: 75000
Employee added.
Enter choice: 1  ID: 102  Name: Rahul Sharma  Dept: HR  Salary: 55000
Employee added.
Enter choice: 2
-----
EMP_ID   NAME                DEPARTMENT    SALARY
-----
101      Prathamesh Yewale   Engineering    75000.00
102      Rahul Sharma        HR             55000.00
-----
Enter choice: 3  ID: 102  New Salary: 65000    Salary updated.
Enter choice: 4  ID: 999  ID 999 not found.
Enter choice: 5  Exiting. Goodbye!

```

Viva Questions & Answers

Q1. What is JDBC and what are its main components?

Ans: JDBC (Java Database Connectivity) is an API for connecting Java applications to databases. Its main components are: DriverManager (manages drivers), Connection (represents a session with DB), Statement/PreparedStatement (executes SQL), and ResultSet (holds query results).

Q2. What is the difference between Statement and PreparedStatement?

Ans: Statement executes plain SQL strings and is prone to SQL injection. PreparedStatement uses parameterized queries with placeholders (?), is precompiled by the database for better performance, and is safe from SQL injection.

Q3. How does PreparedStatement prevent SQL injection?

Ans: PreparedStatement separates SQL logic from data. User input is passed as parameters using setXxx() methods and is treated purely as data — not as SQL code — so malicious input like ' OR 1=1 cannot alter the query structure.

Q4. What is the purpose of ResultSet in JDBC?

Ans: ResultSet holds the rows returned by a SELECT query. It provides a cursor that points to the current row. Methods like rs.next() move to the next row, and rs.getString(), rs.getInt() etc. retrieve column values.

Q5. What is try-with-resources and why is it used with database connections?

Ans: Try-with-resources automatically closes resources (Connection, PreparedStatement, ResultSet) when the try block exits, even if an exception occurs. This prevents resource leaks which could exhaust database connections.

Practical 11: Weather Data Analysis using JavaFX

Aim

To read weather data from a CSV file, perform statistical operations using Apache Commons Math, and visualize it using JavaFX.

Objectives

- Read and parse CSV files using `BufferedReader` and `FileReader`.
- Compute statistics (mean, max, min) using Apache Commons Math library.
- Create a JavaFX `LineChart` to visualize temperature trends over days.
- Integrate file I/O, an external library, and a GUI in one application.

Code

weather.csv

```
Temperature,Humidity
10,74
20,52
45,45
24,78
78,21
```

App.java

```
import java.io.*; import java.util.*;
import javafx.application.Application;
import javafx.scene.Scene; import javafx.scene.chart.*;
import javafx.stage.Stage;
import org.apache.commons.math3.stat.descriptive.DescriptiveStatistics;
public class App extends Application {
    static List<Double> temperature=new ArrayList<>();
    static List<Double> humidity=new ArrayList<>();
    public static void main(String[] args){ readCSV(); launch(args); }
    static void readCSV() {
        try {
            BufferedReader br=new BufferedReader(new FileReader("weather.csv"));
            String line; br.readLine();
            while((line=br.readLine())!=null) {
                String[] data=line.split(",");
                temperature.add(Double.parseDouble(data[0]));
                humidity.add(Double.parseDouble(data[1]));
            }
            DescriptiveStatistics stats=new DescriptiveStatistics();
            for(double t:temperature) stats.addValue(t);
            System.out.println("Average Temp: "+stats.getMean());
            System.out.println("Max Temp: "    +stats.getMax());
            System.out.println("Min Temp: "    +stats.getMin());
        } catch(Exception e){ e.printStackTrace(); }
    }
    @Override
    public void start(Stage stage) {
        CategoryAxis xAxis=new CategoryAxis();
        NumberAxis yAxis=new NumberAxis();
        LineChart<String,Number> chart=new LineChart<>(xAxis,yAxis);
        chart.setTitle("Temperature Trend");
        XYChart.Series<String,Number> series=new XYChart.Series<>();
        for(int i=0;i<temperature.size();i++)
            series.getData().add(new XYChart.Data<>("Day "+(i+1),temperature.get(i)));
        chart.getData().add(series);
        stage.setScene(new Scene(chart,600,400)); stage.show();
    }
}
```

• **Output:**

```
Average Temp: 35.4
Max Temp: 78.0
Min Temp: 10.0

JavaFX Window launched - Title: "Temperature Trend"
Line Chart plotted with 5 data points:
Day 1 -> 10.0 Day 2 -> 20.0 Day 3 -> 45.0 Day 4 -> 24.0 Day 5 -> 78.0
```

Viva Questions & Answers

Q1. What is JavaFX and how does it differ from Swing?

Ans: JavaFX is a modern Java GUI framework using FXML and CSS for UI design, offering rich controls and charts. Swing is an older framework with basic components. JavaFX uses a Scene Graph model and supports animations, while Swing uses a component-based model.

Q2. What does DescriptiveStatistics from Apache Commons Math provide?

Ans: DescriptiveStatistics provides statistical measures over a dataset: mean, max, min, standard deviation, variance, percentiles, skewness, and kurtosis. Values are added using addValue() and retrieved using getMean(), getMax(), getMin() etc.

Q3. What is the role of the launch() method in a JavaFX application?

Ans: launch() is a static method in Application class that initializes the JavaFX runtime, creates an instance of the Application class, and calls the start() method on the JavaFX Application Thread. It must be called from the main thread.

Q4. How does BufferedReader differ from FileReader?

Ans: FileReader reads characters one by one from a file. BufferedReader wraps a FileReader and reads data in chunks into an internal buffer, making it much more efficient. It also provides readLine() for reading complete lines.

Q5. What is the difference between LineChart and BarChart in JavaFX?

Ans: LineChart connects data points with lines, making it ideal for showing trends over time (e.g., temperature). BarChart displays data as rectangular bars, better suited for comparing discrete categories (e.g., humidity per day).